THE PRACTICAL ENGINEERING PLAYBOOK

How Anthropic builds with AI

Briefs, verification, review
and the human decisions that matter.

20 minutes



Based on research dated 27 October 2026

What changed inside Claude Code

~90%

of Claude Code's own code comes from Claude

>2x

merged pull requests

>40%

reduction in the share reverted

Higher output changes the engineering process.
Human attention shifts toward judgment and verification.

Claude Code team self-report [1]. PR and revert comparisons: June to September 2026. Not a controlled study.

The engineering loop

01 **Specify**
A person defines the outcome, constraints and success evidence.

02 **Delegate**
Claude Code or Slack agents implement, test and iterate.

03 **Verify**
Runnable checks and a fresh reviewer test the result.

04 **Approve**
Automated gates and risk determine required human review.

05 **Release**
Flags, staged exposure and monitoring limit the impact.

06 **Learn**
Incidents and recurring findings improve tests and instructions.

Synthesis of documented practices [1, 2, 6]. Teams use different tools and approval policies.

Tasks worth delegating

Strong first candidates

Bug fixes with a reproducible failure

Tests for known behavior

Refactors with stable interfaces

Small features with visible acceptance criteria

Human framing comes first

Ambiguous product decisions

Changes spanning undocumented systems

Migrations with unclear invariants

Work without a reliable correctness check

A good delegation target has a bounded scope and an observable finish line.

Practical selection guide derived from Anthropic team workflows [3].

A brief an agent can execute

Example task: fix duplicate submissions during a slow save

OUTCOME	A repeated click cannot create a second pending save request.
CONTEXT	Inspect the form, save handler and existing integration tests.
CONSTRAINTS	Preserve validation and keyboard access. Keep the API unchanged.
PROOF	Reproduce the bug. Add a failing test, fix it, then rerun the checks.
HANDOFF	Return the diff, test output, remaining risks and rollback steps.
STOP RULE	Ask before changing shared contracts or expanding the scope.

Illustrative brief based on verification and task-framing practices [1, 3].

Proof of completion

Behavior

Show the expected result on the actual user journey.

Checks

Attach relevant test, build or browser output.

Regression

Show that the check catches the original failure.

Limits

State what you did not verify and why.



Adapted from the Claude Code team's executable verification practice [1, 3].

Context that survives the session

`CLAUDE.md`	**Project knowledge** Stack, commands, conventions and non-goals. Keep it short and link to deeper documentation.
`REVIEW.md`	**Review priorities** Describe real failure modes, sensitive areas and evidence required for a finding.
`Skills + checks`	**Repeatable work** Encode useful procedures. Turn recurring feedback into a skill, lint rule or test.

Repeated corrections are evidence that the working context needs an update.

Claude Code team practice [1]. File names and conventions can be adapted to another agent harness.

Parallel work with one owner

Anthropic delegates work to multiple Claudes through Slack.
Parallel work still needs clear interfaces and a person accountable for the result.

Independent work

One agent implements the change.

A fresh reviewer examines the diff.

Other agents validate findings.

Integration responsibility

A person owns shared assumptions and the final change.

Suggested start: one implementer and one reviewer. Expand when coordination works.

Slack delegation and fresh-context review [6, 1]. Starting configuration is a suggested adaptation.

Review that converges

50+

review / fix rounds
on one PR overnight

A documented failure of agents keeping one another busy.

3

rounds before the author must reconsider the approach

Later passes focus on changed code. Reviewers avoid repeating findings. They identify the whole bug class.

Claude Code team example and revised review policy [1]. The three-round cap triggers reassessment.

Human approval follows risk

0

LOW RISK

Docs, copy, internal features or additive changes behind a flag.

1

MEDIUM RISK

Ordinary product changes and sensitive behavior such as caching or system prompts.

2

HIGH RISK

The highest-risk changes and paths with an explicit two-person requirement.

Required human approvals after automated review, security checks and CI pass. Lighter paths apply only to trusted authors. Sensitive paths retain human requirements.

The directing human owns the change. An agent cannot fill a required human slot.

Claude Code team policy [1]. The directing author can count toward the required human total.

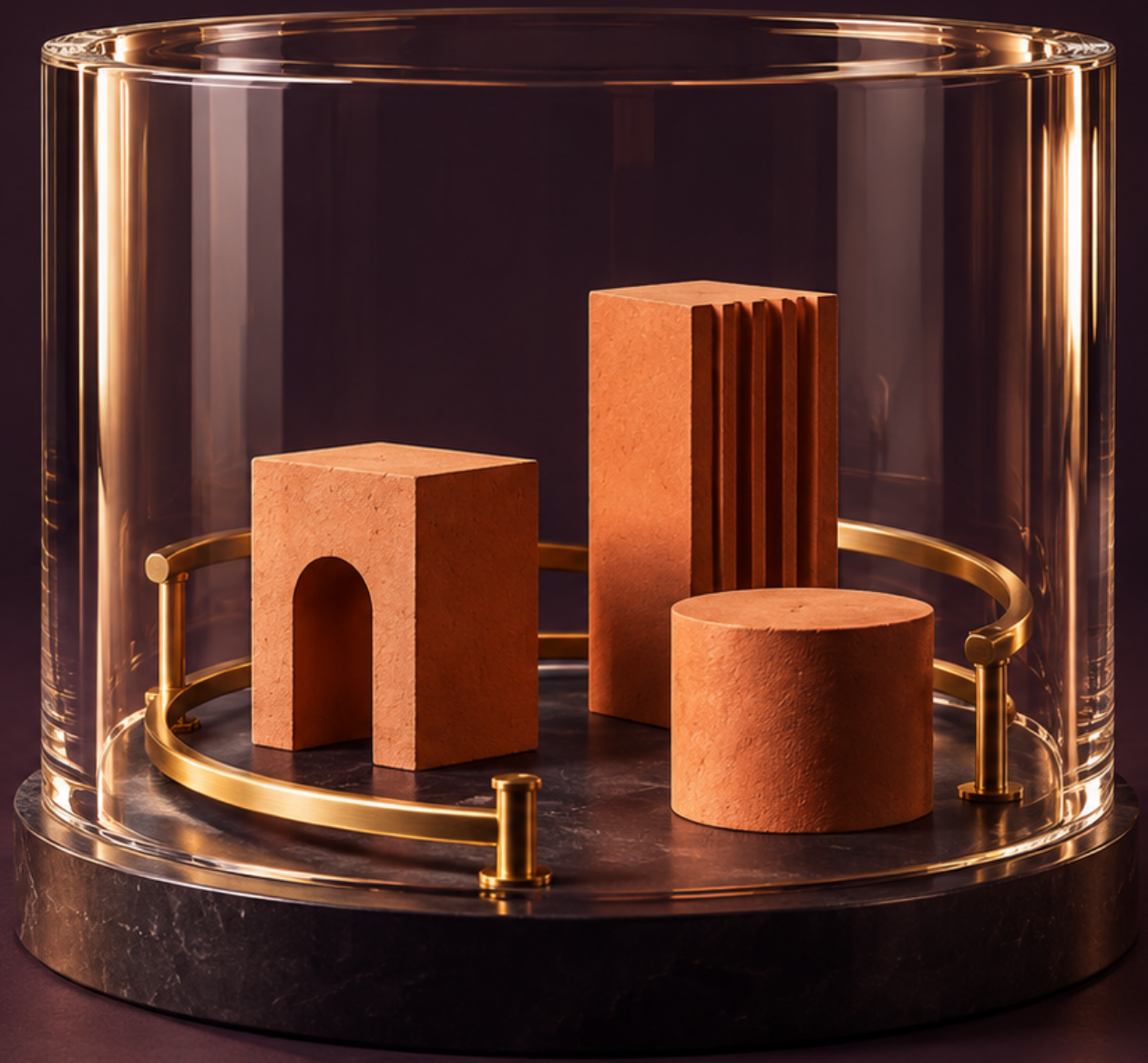
Autonomy needs enforced limits

Workspace
Limit which files the agent can read and change.

Network
Allow only the destinations the task needs.

Identity
Scope credentials, tools, data and spending.

Oversight
Keep action logs and clear escalation boundaries.



Filesystem and network isolation [4]. Scoped access, spend controls and audit logs [6].

A release path built for recovery

01	**Merge queue**	Run checks against main with the change applied.
02	**Feature flag**	Keep new behavior easy to disable.
03	**Staged exposure**	Exercise changes internally before wider release.
04	**Detection**	Watch user behavior, failures and incident signals.
05	**Recovery**	A named owner can disable, revert or fix quickly.

Before merging: how will we notice a failure, and how will we undo the change?

Release and rollback practices reported by the Claude Code team [1].

Prompts and reviewers need regression tests

Past incidents test review

Save the offending PR and its fix or revert.

Confirm the bug with an engineer.

Test whether the reviewer catches it.

Include clean PRs to measure false alarms.

Configuration needs tests

Anthropic traced quality losses to reasoning settings, context caching and a brevity prompt.

Its response included broader model-specific evaluations, the public build in internal testing, and gradual rollout.

Incident-driven reviewer evaluations [1]. April 2026 quality postmortem [5].

Humans own system-level judgment

A locally plausible change can still violate the architecture.

Scion's source-control team

Agents sometimes scanned commit history when the code-review service already owned the answer.

Adversarial review sometimes added excessive validation for hypothetical concerns.

What the human decides

Which system owns this fact?

Which invariants must stay true?

Is the added complexity justified?

Are we solving the right problem?

Scion team examples [2]. Accountability remains with the person directing the agent [1].

Bottlenecks move beyond coding

Anthropic monorepo: merged changes per day

~2,500

May 2026

>30,000

September 2026

Seven engineers built Scion source control, Antique code review and related systems, then migrated the monorepo in five months.

Transferable lesson

Measure review queues, checkouts and CI. Improve the limiting stage before adding more agent capacity.

Anthropic self-report [2]. Scion is internal and tailored to an unusual workload, not a general Git replacement.

A scorecard for useful throughput

Delivery	Time from accepted task to verified change
Human effort	Review time, waiting and repair work
Quality	Escaped defects, reverts and incidents
Review efficiency	False alarms and rounds to pass
Economics	Total cost per accepted change

Expand autonomy when delivery improves and quality holds.

Suggested scorecard. Review time, reverts, incidents, noise and convergence reflect reported practice [1].

A two-week pilot

One repository. One owner. A small set of comparable tasks.

DAYS 1-2	**Establish the baseline** Choose bounded tasks, record current delivery and review effort, and define quality limits.
DAYS 3-5	**Run the complete loop** Use the brief, runnable proof and fresh review. Keep required human approvals.
DAYS 6-10	**Improve one bottleneck** Turn repeated feedback into context or checks. Compare results and total effort with baseline.

Decision: expand one task type, refine the process, or pause if quality worsens.

Proposed adoption plan derived from the research [1, 3]. It is not Anthropic's published rollout schedule.

The next task is the starting point

01 Write the outcome and constraints.

02 Define the proof before implementation.

03 Name the owner, review rule and rollback.

Engineering judgment and verification make delegation useful.

Practical synthesis of Anthropic's reported workflows [1, 2, 3].

Sources for the playbook

[1] **How the Claude Code team does code review**

15 October 2026

Verification, review, approvals, release controls, convergence and the Claude Code metrics.

[2] **Redesigning source control at Anthropic**

13 October 2026, updated 15 October

Scion's scale, delivery timeline, architectural tradeoffs and human oversight.

[3] **How Anthropic teams use Claude Code**

24 July 2025

Team workflows for testing, debugging, code navigation and bounded implementation.

Reference slide. Titles are clickable links to the primary sources. Research cutoff: 27 October 2026.

Sources and evidence limits

[4] **Making Claude Code more secure and autonomous**

20 October 2025

[5] **An update on recent Claude Code quality reports**

23 April 2026

[6] **Introducing Claude Tag**

23 June 2026

How to read the evidence

Metrics are Anthropic self-reports, with different teams and measurement periods. They do not establish causal productivity gains. The example brief, scorecard and pilot are proposed adaptations. Public disclosures do not establish one universal internal setup.

Based on the supplied research and linked primary sources. Reference slides are outside the 20-minute pacing.